

SCX 通用模块化架构设计

日期：2026-06-26 来源：基于 GPT 对话分析（与 `gpt` 的对话 202606261332.md）目标：解决”每加一个领域就要写新的 adapter，代码重复，接口不一致”的问题

目录

1. 架构总览（ASCII 图）
 2. 设计原则
 3. 核心抽象接口定义（ABC）
 4. SCX Core（不变层）
 5. SCX Encoder（领域相关，但接口统一）
 6. SCX Adapter（自动生成）
 7. 声明式领域配置（YAML/JSON）
 8. 文件夹结构
 9. SCX Domain 注册与生命周期
 10. 插件系统（可选扩展）
 11. 与现有代码的迁移路径
 12. 新增领域的步骤（3 步）
 13. 数据流全景
 14. 附录：领域配置参考
-

1. 架构总览

SCX Universal Core

StateSpace	ExpertRelia-	DataClassi-	ActionPolicy
$S = \{s_1 \dots K\}$	bility	fier	$a^*(s)$
$R_m(s)$	$R_m(s)$, SCX	4-classify	keep/route/
$V(s)$	$\rightarrow$ expert gap	V/R/N/E	compress/...

SCX State-Outcome Database

(s, e, a, o) — persistent across domains

SCX Encoder
(plug per domain)

SCX Domain Config
(YAML declaration)

$encode(x) \rightarrow z$	encoder: path
$distance(a,b) \rightarrow d$	experts: [...]
$cluster(Z) \rightarrow s_k$	state_config: {...}
	thresholds: {...}

Domain Adapter
(auto-generated)

Encoder → StateSpace
→ ExpertReliability
→ DataClassifier

Domain-specific (per industry):

MLIP	Health	CIFAR	Robot	FEM
ACE/SOAP	ResNet	CNN	Trajectory	Mesh

Plugins (optional):

SCX-	SCX-	SCX-
Compress	Online	Influence

层间依赖规则

SCX Core	← 不依赖任何 domain 代码
SCX Encoder	← 依赖 Core (实现抽象接口), 不依赖其他 domain
SCX Domain Config	← 声明式, 不依赖 Python 代码
SCX Adapter	← 自动生成, 桥接 Encoder 和 Core
SCX Plugins	← 可选, 可依赖 Core + Encoder + Adapter

核心原则：Core 永远不知道任何 Encoder 或 Domain 的存在。所有领域知识通过 Encoder 接口 + Domain 配置注入。

2. 设计原则

2.1 通用数学层

SCX 的数学核心不依赖任何领域概念：

数学量	符号	与领域无关的定义
状态空间	$S = \{s, \dots, s_K\}$	表示空间的聚类划分
专家期望风险	$R_m(s) = E[(f_m(x), f^*(x)) \mid x \in s]$	任意损失函数下的条件期望
专家可靠性	$SCX_m(s) = P(\cdot < \cdot \mid s)$	阈值条件下的可接受概率
数据价值	$V(s) = (s) \cdot [1-D(s)] \cdot A_{\{e^*\}}(s) \cdot W_{\{weak\}}(s) \cdot [1-N(s)]$	五维乘积

数学量	符号	与领域无关的定义
数据分类	{Valuable, Redundant, Noisy, ExpertDependent}	基于阈值的四分类
动作策略	$a^*(s) = \operatorname{argmax}_a E[U(a,s,e,o)] - C(a)$	成本约束的效用最大化

2.2 控制反转 (IoC)

领域代码不直接调用 Core。Core 通过抽象接口调用领域代码 (Encoder)。

传统方式: Domain → Core (领域代码主动调用框架)
SCX 方式: Core → Encoder (框架通过抽象接口回调领域代码)

2.3 配置驱动而非代码驱动

添加新领域不需要写 Python 集成代码, 只需要: 1. 写一个 Encoder 类 (实现接口) 2. 写一个 YAML 配置文件 3. 注册到 domain registry

3. 核心抽象接口定义

3.1 SCXStateEncoder

```
# scx/encoders/base.py
```

```
from abc import ABC, abstractmethod
from typing import Any, Optional
import numpy as np
```

```
class SCXStateEncoder(ABC):
```

```
    """ 将任意领域输入  $x$  编码为状态空间中的向量。
```

```
    所有领域 (MLIP, Health, CIFAR, Robot, FEM, Semi, Text...)
```

```
    只需实现这三个方法, 即可接入 SCX 完整框架。
```

```
    """
```

```
@abstractmethod
```

```
def encode(self, x: Any) -> np.ndarray:
```

```
    """ 将原始输入  $x$  映射为稠密向量  $z \in \mathbb{R}^d$ 。
```

```
    这是状态编码的核心。可以是:
```

- *MLIP: ACE/SOAP descriptor*
- *Vision: CNN/ViT feature embedding*
- *Trajectory: temporal encoding of (o, a, r) sequence*
- *Tabular: normalized + embedded feature vector*
- *Text: LLM embedding*
- *Simulation: mesh/grid encoding*

Parameters

x : Any

单个原始输入样本。

Returns

np.ndarray, shape (d,)

编码后的稠密向量。

"""

...

```
def encode_batch(self, X: list[Any]) -> np.ndarray:
```

""" 批量编码，默认逐个调用 *encode()*，子类可重写做并行加速。 """

```
    return np.array([self.encode(x) for x in X])
```

@abstractmethod

```
def distance(self, a: np.ndarray, b: np.ndarray) -> float:
```

""" 定义状态空间中的距离度量。

默认可以用欧氏距离，但不同领域可能需要自定义：

- SOAP 特征用 *cosine* 距离
- 轨迹用 *DTW*
- 图结构用 *graph edit distance*

Parameters

a : np.ndarray, shape (d,)

```

        b : np.ndarray, shape (d,)

    Returns
    -----
    float
        距离（越小越近）。
    """
    ...

@abstractmethod
def cluster(self, Z: np.ndarray, **kwargs) -> tuple[np.ndarray, np.ndarray]:
    """ 对编码后的向量集进行聚类，划分状态。

    Parameters
    -----
    Z : np.ndarray, shape (N, d)
        编码向量矩阵。

    Returns
    -----
    labels : np.ndarray, shape (N,)
        每个样本的状态分配  $\{0, \dots, K-1\}$ 。
    centroids : np.ndarray, shape (K, d)
        每个状态的质心。
    """
    ...

def get_feature_dim(self) -> int:
    """ 返回编码向量的维度  $d$ 。 """

```



```

    if hasattr(self, '_feature_dim'):
        return self._feature_dim
    # 尝试通过一个 dummy 输入推断
    dummy = self.encode(self._dummy_input()) if hasattr(self, '_dummy_input')
    if dummy is not None:
        return dummy.shape[0]
    return 0

def _dummy_input(self) -> Any:
    """ 子类可重写以提供用于推断维度的模拟输入。 """
    raise NotImplementedError

```

3.2 SCXExpert（轻量抽象）

```

# scx/core/expert.py

from abc import ABC, abstractmethod
from typing import Any

class SCXExpert(ABC):
    """ 任意专家：ML 模型、人、仿真器、API、规则系统。

    核心思想：SCX 不关心专家内部实现，只关心：
    - predict(x) → y: 预测输出
    - cost() → float: 使用成本（用于成本约束决策）
    """

    @abstractmethod

```

```

def predict(self, x: Any) -> Any:
    """ 对单个输入  $x$  做出预测/决策。

    Parameters
    -----
     $x$  : Any
        输入样本。

    Returns
    -----
    Any
        专家输出（可以是标量、向量、类别、文本等）。
    """
    ...

def predict_batch(self, X: list[Any]) -> list[Any]:
    """ 批量预测，默认逐个调用。 """
    return [self.predict(x) for x in X]

@abstractmethod
def cost(self) -> float:
    """ 返回使用该专家一次的成本（相对值，用于路由决策）。

    例如：
    - 轻量模型：0.1
    - DFT 计算：100.0
    - 人类专家：500.0
    - 高保真仿真：50.0
    """

```

```

...

@property
def name(self) -> str:
    """ 专家名称，用于日志和报告。 """
    return self.__class__.__name__

@property
def metadata(self) -> dict:
    """ 专家元数据（可选的扩展信息）。 """
    return {}

```

3.3 SCXDataPoint（可选抽象）

```

# scx/core/datapoint.py

from abc import ABC, abstractmethod
from typing import Any
import numpy as np

class SCXDataPoint(ABC):
    """ 任意数据点：DFT 帧、图像、轨迹片段、仿真结果、文本。

    方便封装常见数据操作。非必须——可以直接传原始类型给 Encoder。
    """

    @abstractmethod
    def get_features(self) -> np.ndarray:

```

```
        """ 返回该数据点的特征向量（用于编码前处理）。 """
        ...

    def get_label(self) -> Any:
        """ 返回该数据点的标签（如果可用）。 """
        return None

    @classmethod
    @abstractmethod
    def from_raw(cls, raw: Any) -> "SCXDataPoint":
        """ 从原始格式创建数据点。 """
        ...
```

4. SCX Core（不变层）

Core 层完全保持通用，不引用任何领域代码。它包括：

4.1 现有模块（保留，不重写）

模块	文件	说明
SCXFramework	core/framework.py	顶层编排类，重构为接收 Encoder + Config
SCXConfig	core/config.py	扩展为支持 domain config 加载
StateSpace	state/space.py	状态空间容器（不变）

模块	文件	说明
StateDiscovery	state/discovery.py	状态发现（接收 Encoder.cluster）
ExpertReliability	expert/reliability.py	专家可靠性估计（不变）
ExpertRegistry	expert/registry.py	专家注册（不变）
DataClassifier	valuation/classifier.py	网分类（不变）
ActionPolicy	action/policy.py	动作策略（不变）
OnlineSCXFramework	core/online.py	在线监测（不变）

4.2 新增 Core 模块

SCXDomainEngine

```
# scx/core/domain_engine.py

class SCXDomainEngine:
    """ 领域引擎：将 Encoder + Config + Core 串联的胶水层。

    这个类是用户的主要入口点。替代原来直接使用 SCXFramework。
    """

    def __init__(self, domain_config: dict | str):
        # 加载 YAML/JSON 配置
        # 动态实例化 Encoder
        # 创建 ExpertRegistry
        # 配置阈值
        ...
```

```
def fit(self, X, y=None):
    """ 运行完整 SCX pipeline。 """

def audit(self) -> "SCXAuditReport":
    """ 生成数据价值审计报告。 """

def value(self, x) -> float:
    """ 计算单个样本的数据价值。 """

def recommend_action(self, state_id) -> str:
    """ 推荐动作。 """

def save(self, path): ...
def load(self, path): ...
```

SCXAuditReport

```
# scx/core/audit.py
```

```
class SCXAuditReport:
```

```
    """ 标准审计报告，跨领域统一格式。
```

```
    报告包含：
```

1. 状态覆盖地图
2. 冗余比例
3. 噪声风险
4. 高价值状态
5. *Expert* 可靠性矩阵

```
6. 建议动作
7. 执行后收益估计
"""
```

5. SCX Encoder（领域相关，但接口统一）

5.1 内置 Encoder 列表

```
scx/encoders/
    base.py          # SCXStateEncoder 抽象基类
    mlip.py          # ACE/SOAP descriptor → 状态向量
    vision.py        # CNN/ViT/CLIP → 图像 embedding
    tabular.py       # 表格数据 → 归一化向量
    trajectory.py    # 机器人轨迹 → temporal encoding
    simulation.py    # FEM/PDE mesh → response encoding
    text.py          # LLM embedding → 语义向量
    health.py        # 医学图像 → ResNet feature
```

5.2 Encoder 示例

MLIP Encoder

```
# scx/encoders/mlip.py

class ACEEncoder(SCXStateEncoder):
    """ACE/SOAP descriptor 编码器，用于原子构型。"""
```

```

def __init__(self, descriptor: str = "soap", rcut: float = 5.0, nmax: int = 8
            self.descriptor_type = descriptor
            self.rcut = rcut
            self.nmax = nmax
            self.lmax = lmax
            # lazily initialize describe/etc. here

def encode(self, x: "ase.Atoms") -> np.ndarray:
    # x 是 ASE Atoms 对象
    # 返回 SOAP/ACE 描述符向量
    ...

def distance(self, a: np.ndarray, b: np.ndarray) -> float:
    # SOAP 常用 cosine 距离
    return 1.0 - np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-10)

def cluster(self, Z: np.ndarray, **kwargs) -> tuple[np.ndarray, np.ndarray]:
    # 用 HDBSCAN 或 K-Means 聚类
    from sklearn.cluster import KMeans
    n = kwargs.get("n_states", 10)
    km = KMeans(n_clusters=n, random_state=42)
    labels = km.fit_predict(Z)
    return labels, km.cluster_centers_

```

Vision Encoder

```
# scx/encoders/vision.py
```

```
class CNNEncoder(SCXStateEncoder):
```



```
"""CNN/ViT encoder for images."""
```

```
def __init__(self, backbone: str = "resnet18", device: str = "cpu"):
    import torch
    self.device = device
    self.model = self._build_backbone(backbone).to(device)
    self.model.eval()

def encode(self, x: "torch.Tensor | np.ndarray") -> np.ndarray:
    import torch
    if isinstance(x, np.ndarray):
        x = torch.from_numpy(x).to(self.device)
    if x.ndim == 3:
        x = x.unsqueeze(0) # add batch dim
    with torch.no_grad():
        feat = self.model.forward_features(x)
    return feat.cpu().numpy().flatten()

def distance(self, a: np.ndarray, b: np.ndarray) -> float:
    return float(np.linalg.norm(a - b))

def cluster(self, Z: np.ndarray, **kwargs) -> tuple[np.ndarray, np.ndarray]:
    from sklearn.cluster import MiniBatchKMeans
    n = kwargs.get("n_states", 20)
    km = MiniBatchKMeans(n_clusters=n, random_state=42)
    labels = km.fit_predict(Z)
    return labels, km.cluster_centers_
```

Trajectory Encoder

```
# scx/encoders/trajectory.py
```

```
class TrajectoryEncoder(SCXStateEncoder):
    """ 机器人轨迹编码器。 """

    def __init__(self, state_dim: int = 128, use_temporal: bool = True):
        self.state_dim = state_dim
        self.use_temporal = use_temporal
        # lazily build encoder network

    def encode(self, x: dict) -> np.ndarray:
        # x 包含 observation, action, reward, success 等
        # 返回固定维度的轨迹 embedding
        ...

    def distance(self, a: np.ndarray, b: np.ndarray) -> float:
        # 轨迹用 DTW 或简单欧氏距离
        return float(np.linalg.norm(a - b))

    def cluster(self, Z: np.ndarray, **kwargs) -> tuple[np.ndarray, np.ndarray]:
        from sklearn.cluster import HDBSCAN
        clusterer = HDBSCAN(min_cluster_size=kwargs.get("min_cluster_size", 5))
        labels = clusterer.fit_predict(Z)
        # HDBSCAN 可能产生 -1 (noise), 需要处理
        centroids = ...
        return labels, centroids
```

6. SCX Adapter（自动生成）

Adapter 是 Encoder 和 Core 之间的桥接层。它从 Encoder 和 Domain Config 自动推导出 Core 所需的全部组件。

6.1 自动适配的内容

组件	来源	自动适配逻辑
StateSpace	Encoder.cluster()	从 labels + centroids 构建结果
Expert 列表	Domain Config experts 字段	根据 type 自动实例化专家
损失函数	默认 MSE 或 Config 指定	根据问题类型选择
R_m(s)	ExpertReliability.getState()	使用 Encoder 的 state assignment
阈值	Domain Config thresholds	覆盖 Core 默认值
特征维度	Encoder.get_features()	自动推断
聚类方法	Encoder.cluster()	直接委托

6.2 Adapter 生成伪代码

```
# scx/core/adapter.py

def build_from_domain(domain_config: dict) -> SCXDomainEngine:
    """ 从领域配置自动构建 SCXDomainEngine。 """

    # 1. 实例化 Encoder
```

```

encoder_cls = import_class(domain_config["encoder"])
encoder = encoder_cls(**domain_config.get("encoder_params", {}))

# 2. 创建 ExpertRegistry
registry = ExpertRegistry()
for exp_cfg in domain_config.get("experts", []):
    expert = create_expert(exp_cfg) # 根据 type 工厂方法
    registry.register(expert)

# 3. 构建 SCXConfig (叠加领域阈值)
base_config = SCXConfig()
overrides = domain_config.get("thresholds", {})
for k, v in overrides.items():
    if hasattr(base_config, k):
        setattr(base_config, k, v)
config = base_config

# 4. 构建领域引擎
engine = SCXDomainEngine(config=config, encoder=encoder, registry=registry)
return engine

```

6.3 Encoder $\rightarrow$ StateSpace 的自动推导

```

# SCXDomainEngine.fit() 内部

def fit(self, X, y=None):
    # 1. Encode
    Z = self.encoder.encode_batch(X)

```

```
# 2. Cluster → states
labels, centroids = self.encoder.cluster(Z, n_states=self.config.n_states)

# 3. Build StateSpace
self.state_space = StateSpace(n_states=len(centroids))
self.state_space.assignment = labels
for k, centroid in enumerate(centroids):
    mask = labels == k
    info = StateInfo(
        id=k,
        label=f"state_{k}",
        centroid=centroid,
        radius=float(np.mean([self.encoder.distance(centroid, Z[i])
                               for i in np.where(mask)[0]])),
        count=int(mask.sum()),
        proportion=float(mask.mean()),
    )
    self.state_space.add_state(info)

# 4. Expert reliability
self.expert_reliability = ExpertReliability(...)
result = self.expert_reliability.estimate(
    registry=self.expert_registry,
    X=X, y=y,
    state_assignments=labels,
    n_states=len(centroids),
)

# 5. Data classification
```

```
self.classifier = DataClassifier(config=...)
...

# 6. Action policy
...
```

7. 声明式领域配置

每个领域只需一个 YAML 文件：

7.1 MLIP 领域配置

```
# domains/mlip.yaml
domain: mlip
description: " 机器学习原子间势函数的数据价值评估与专家路由"

encoder:
  class: scx.encoders.mlip.ACEEncoder
  params:
    descriptor: soap
    rcut: 5.0
    nmax: 8
    lmax: 6

experts:
  - name: ACE
    type: model
```

```
    predict_fn: "path/to/ace_model.ace"
    cost: 0.5
- name: NEP
  type: model
  predict_fn: "path/to/nep_model.txt"
  cost: 0.3
- name: MACE
  type: model
  predict_fn: "path/to/mace_model.pt"
  cost: 1.0
- name: DFT
  type: simulation
  command: "vasp_std"
  cost: 100.0

state_config:
  n_states: 20
  cluster_method: hdbscan
  min_cluster_size: 10

thresholds:
  error_high: 0.05          # eV/atom
  density_high: 0.05
  consistency_high: 0.7
  redundancy_high: 0.8
  noise_high: 0.5
  expert_gap: 0.3

loss_fn: mse                # 或 "mae", "rmse"
```

```
reliability:
  method: supervised
  alpha: 1.0
  min_samples: 5
```

```
actions:
  - acquire
  - compress
  - route
  - high_fidelity
```

7.2 Health 领域配置

```
# domains/health.yaml
domain: health_image
description: " 医学图像数据价值审计与专家复核调度"

encoder:
  class: scx.encoders.vision.CNNEncoder
  params:
    backbone: resnet18
    device: cuda

experts:
  - name: ai_model_v1
    type: model
    checkpoint: "checkpoints/v1.pth"
    cost: 0.1
```



```
- name: ai_model_v2
  type: model
  checkpoint: "checkpoints/v2.pth"
  cost: 0.2
- name: junior_doctor
  type: human
  cost: 50.0
- name: senior_doctor
  type: human
  cost: 200.0

state_config:
  n_states: 30
  cluster_method: kmeans
  random_state: 42

thresholds:
  error_high: 0.3
  density_high: 0.02
  consistency_high: 0.6
  redundancy_high: 0.85
  noise_high: 0.4
  expert_gap: 0.25

loss_fn: cross_entropy

reliability:
  method: hybrid
  alpha: 2.0
```

```
actions:
  - keep
  - compress
  - route
  - relabel
```

7.3 Robot 领域配置

```
# domains/robot.yaml
domain: robot_trajectory
description: " 机器人示范轨迹价值评估与专家策略路由"

encoder:
  class: scx.encoders.trajectory.TrajectoryEncoder
  params:
    state_dim: 128
    use_temporal: true

experts:
  - name: scripted_policy
    type: rule_based
    cost: 0.01
  - name: rl_policy
    type: model
    checkpoint: "checkpoints/rl.pt"
    cost: 0.5
  - name: mpc_controller
    type: simulation
```

```
    command: "mpc_solver"
    cost: 10.0
  - name: human_teleop
    type: human
    cost: 500.0
  - name: vla_model
    type: model
    cost: 2.0

state_config:
  n_states: 20
  cluster_method: hdbscan
  min_cluster_size: 5

thresholds:
  error_high: 0.15
  noise_high: 0.5
  redundancy_high: 0.75
  expert_gap: 0.2

reliability:
  method: bayesian
  min_samples: 3

actions:
  - keep
  - compress
  - route
  - high_fidelity          # 上真实机器人
```

- acquire

7.4 FEM/Simulation 领域配置

```
# domains/fem.yaml
domain: fem_simulation
description: " 有限元仿真多保真调度与数据压缩"

encoder:
  class: scx.encoders.simulation.FEMEncoder
  params:
    mesh_encoder: "pointnet"
    response_dim: 64

experts:
  - name: coarse_mesh
    type: simulation
    config: "configs/coarse.json"
    cost: 1.0
  - name: fine_mesh
    type: simulation
    config: "configs/fine.json"
    cost: 50.0
  - name: neural_operator
    type: model
    checkpoint: "checkpoints/don.pt"
    cost: 0.1
  - name: analytical
    type: rule_based
```

```
    cost: 0.01

state_config:
  n_states: 15
  cluster_method: spectral

thresholds:
  error_high: 0.1
  redundancy_high: 0.8
  noise_high: 0.4
  expert_gap: 0.3

actions:
  - keep
  - compress
  - route
  - high_fidelity
```

7.5 Tabular/Benchmark 配置

```
# domains/cifar.yaml
domain: cifar_image
description: "CIFAR 图像数据价值评估"

encoder:
  class: scx.encoders.vision.CNNEncoder
  params:
    backbone: simple_cnn
    device: cpu
```

```
experts:
  - name: cnn_small
    type: model
    cost: 0.1
  - name: cnn_large
    type: model
    cost: 0.5
  - name: resnet50
    type: model
    cost: 1.0

state_config:
  n_states: 25
  cluster_method: kmeans
  random_state: 42

thresholds:
  error_high: 0.3
  density_high: 0.04
  consistency_high: 0.6
  redundancy_high: 0.85
  noise_high: 0.5
  expert_gap: 0.2

loss_fn: cross_entropy

actions:
  - keep
```

- compress
- route
- relabel

8. 文件夹结构

```

scx/
  core/
    __init__.py          # 通用核心（已有，基本不变）
    framework.py         # SCXFramework（重构为领域引擎调度）
    config.py            # SCXConfig（扩展领域配置加载）
    metrics.py           # SCXMetrics（不变）
    online.py            # OnlineSCXFramework（不变）
    domain_engine.py     # SCXDomainEngine（新入口）
    adapter.py           # 自动推导 Adapter
    audit.py             # SCXAuditReport
    expert.py            # SCXExpert 抽象
    datapoint.py         # SCXDataPoint 抽象

  state/
    space.py             # 状态管理（已有，不变）
    discovery.py         # StateSpace + StateInfo
    assignment.py        # 状态发现（不变）
    robustness.py        # 状态分配（不变）
    robustness.py        # 鲁棒性分析（不变）

  expert/
    registry.py          # 专家管理（已有，不变）
    registry.py          # ExpertRegistry（不变）

```

reliability.py	# ExpertReliability (不变)
router.py	# ExpertRouter (不变)
conflict.py	# ExpertConflict (不变)
valuation/	# 数据估值 (已有, 不变)
classifier.py	# DataClassifier (不变)
state_value.py	# 状态价值 (不变)
redundancy.py	# 冗余度 (不变)
noise_score.py	# 噪声评分 (不变)
learnability.py	# 可学习性 (不变)
influence.py	# Influence 分数 (不变)
adaptive.py	# 自适应估值 (不变)
action/	# 动作策略 (已有, 不变)
policy.py	# ActionPolicy (不变)
compress.py	# 压缩策略 (不变)
acquisition.py	# 获取策略 (不变)
encoders/	# 领域编码器 (统一接口)
__init__.py	
base.py	# SCXStateEncoder 抽象基类
mlip.py	# ACE/SOAP descriptor encoder
vision.py	# CNN/ViT/CLIP encoder
tabular.py	# 表格数据 encoder
trajectory.py	# 机器人轨迹 encoder
simulation.py	# FEM/PDE 仿真 encoder
text.py	# LLM/NLP embedding encoder
health.py	# 医学图像 encoder (从 scx-health 迁移)


```
domains/                                # 领域配置（声明式）
    __init__.py
    mlip.yaml
    health.yaml
    cifar.yaml
    robot.yaml
    fem.yaml
    tabular.yaml
    text.yaml
    registry.py                          # 领域注册表

plugins/                                # 插件（可选扩展）
    __init__.py
    compress.py                         # SCX-Compress 插件
    online.py                          # Online SCX 插件
    influence.py                       # Influence 插件

utils/                                  # 工具（已有，扩展）
    helpers.py                         # 辅助函数
    data_loader.py                    # 数据加载
    visualization.py                 # 可视化
    evaluation.py                    # 评估工具

__init__.py
```

9. SCX Domain 注册与生命周期

9.1 领域注册表

```
# scx/domains/registry.py
```

```
class DomainRegistry:
```

```
    """ 全局领域配置注册表，支持按名称查找。 """
```

```
    _domains: dict[str, dict] = {}
```

```
    @classmethod
```

```
    def register(cls, name: str, config: dict | str):
```

```
        """ 注册一个领域。config 可以是 dict 或 YAML 文件路径。 """
```

```
        if isinstance(config, str):
```

```
            import yaml
```

```
            with open(config) as f:
```

```
                config = yaml.safe_load(f)
```

```
        cls._domains[name] = config
```

```
    @classmethod
```

```
    def get(cls, name: str) -> dict:
```

```
        if name not in cls._domains:
```

```
            raise KeyError(f"Domain '{name}' not registered. "
```

```
                           f"Available: {list(cls._domains.keys())}")
```

```
        return cls._domains[name]
```

```
    @classmethod
```

```
    def list_domains(cls) -> list[str]:
```

```
        return list(cls._domains.keys())

    @classmethod
    def load_all(cls, domains_dir: str = None):
        """ 从目录加载所有 YAML 领域配置。 """
        if domains_dir is None:
            domains_dir = Path(__file__).parent
        import glob
        for yaml_file in glob.glob(str(domains_dir / "*.yaml")):
            name = Path(yaml_file).stem
            cls.register(name, yaml_file)

# 启动时自动加载所有领域配置
DomainRegistry.load_all()
```

9.2 生命周期

初始化:

1. DomainRegistry.get("mlip") → 加载 YAML
2. SCXDomainEngine(domain_config) → 构建引擎
3. 自动实例化 Encoder
4. 自动创建 ExpertRegistry
5. 自动配置阈值

运行:

1. engine.fit(X, y) → 完整 pipeline
2. engine.audit() → 输出审计报告
3. engine.value(x) → 单样本价值
4. engine.recommend_action(state_id)

持久化：

1. `engine.save("my_project.scx")` → 序列化全部状态
2. `SCXDomainEngine.load("my_project.scx")` → 恢复

在线模式：

1. `OnlineSCXFramework(initial_centroids, M)` → 流式处理
 2. `process_sample(x, expert_id, loss)` → 增量更新
-

10. 插件系统

插件是可选的扩展模块，提供超越 Core 的额外功能。

10.1 插件接口

```
# scx/plugins/base.py
```

```
class SCXPlugin(ABC):
    """SCX 插件基类。"""

    @abstractmethod
    def name(self) -> str: ...

    @abstractmethod
    def attach(self, engine: SCXDomainEngine) -> None:
        """将插件挂载到领域引擎上。"""
    ...
```

```
def on_fit_complete(self, engine: SCXDomainEngine) -> None:
    """fit() 完成后回调。"""
    ...

def on_classify_complete(self, engine: SCXDomainEngine) -> None:
    """分类完成后回调。"""
    ...
```

10.2 内置插件

插件	文件	功能
CompressPlugin	plugins/compress.py	数据压缩策略（corest selection）
OnlinePlugin	plugins/online.py	流式在线监测
InfluencePlugin	plugins/influence.py	Influence function 数据估值

10.3 插件的使用

```
# 通过配置启用插件
engine = SCXDomainEngine("domains/mlip.yaml")
engine.attach_plugin(OnlinePlugin(tau=0.5, decay=0.95))
engine.fit(X, y)
```

11. 与现有代码的迁移路径

11.1 迁移原则

1. 不破坏现有代码：已有的 `scx.core`, `scx.state`, `scx.expert`, `scx.valuation`, `scx.action` 不变
2. 增量重构：先加新结构，再逐步迁移旧的 domain 代码
3. 向后兼容：SCXFramework 保留，但标记为 deprecated，推荐使用 SCXDomainEngine

11.2 迁移步骤

Phase 1: 新增结构（无破坏）

Step 1: 创建 <code>scx/encoders/base.py</code>	# SCXStateEncoder ABC
Step 2: 创建 <code>scx/encoders/mlip.py</code>	# MLIP Encoder
Step 3: 创建 <code>scx/encoders/vision.py</code>	# Vision Encoder
Step 4: 创建 <code>scx/core/domain_engine.py</code>	# SCXDomainEngine
Step 5: 创建 <code>scx/core/adapter.py</code>	# 自动推导逻辑
Step 6: 创建 <code>scx/core/audit.py</code>	# 审计报告
Step 7: 创建 <code>scx/domains/registry.py</code>	# 领域注册表
Step 8: 创建配置文件 <code>mlip.yaml</code> , <code>health.yaml</code> , <code>cifar.yaml</code>	

Phase 2: 迁移 `scx-health`

当前: `scx-health/src/scx_health/encoder.py` (独立包)

目标: `scx/encoders/health.py` (统一接口)

迁移:

1. 将 SimpleCNN, ResNetEncoder 移植到 `scx/encoders/health.py`

2. 实现 SCXStateEncoder 接口 (encode/distance/cluster)
3. 创建 scx/domains/health.yaml
4. 验证原 scx-health 实验仍能运行
5. 原 scx-health 包保留为 legacy, 标记 deprecated

Phase 3: 迁移 CIFAR 实验

当前: experiments/cifar/utils.py (独立 encoder)

目标: scx/encoders/vision.py (复用统一接口)

迁移:

1. CIFAR 的 SimpleCNN 可以用 scx/encoders/vision.CNNEncoder
2. 创建 scx/domains/cifar.yaml
3. 实验脚本改为使用 SCXDomainEngine
4. 原 experiments/cifar/ 保留, 标记为 legacy

Phase 4: 新领域接入

机器人: scx/encoders/trajectory.py + domains/robot.yaml

FEM: scx/encoders/simulation.py + domains/fem.yaml

表格: scx/encoders/tabular.py + domains/tabular.yaml

文本: scx/encoders/text.py + domains/text.yaml

11.3 新老接口对比

场景	旧方式	新方式
创建框架	SCXFramework(config)	SCXDomainEngine("domains/mlip.yaml")

场景	旧方式	新方式
添加领域	写 adapter 类，修改 framework	写 Encoder + YAML，直接注册
配置阈值	硬编码在 config.py	YAML 中声明
数据编码	调用方自己处理	Encoder.encode() 统一接口
聚类	调用方自己选择算法	Encoder.cluster() 统一接口
特征	手动传递 phi(X)	encoder.encode_batch(X) 自动
审计报告	不统一	SCXAuditReport 标准格式
领域管理	无	DomainRegistry

12. 新增领域的步骤

12.1 标准三步流程

- 第 1 步 写 Encoder
- 第 2 步 写 YAML
- 第 3 步 跑测试

12.2 详细步骤

第 1 步：写 Encoder（约 30-100 行）


```
# scx/encoders/my_domain.py
```

```
from scx.encoders.base import SCXStateEncoder
import numpy as np
```

```
class MyDomainEncoder(SCXStateEncoder):
```

```
    """ 我的领域编码器。 """
```

```
    def __init__(self, param1: int = 10, param2: str = "default"):
        self.param1 = param1
        self.param2 = param2
        self._feature_dim = param1
```

```
    def encode(self, x: Any) -> np.ndarray:
```

```
        # 1. 预处理  $x$ 
        # 2. 提取特征向量
        # 3. 返回  $(d,)$  的 np.ndarray
        ...
```

```
    def distance(self, a: np.ndarray, b: np.ndarray) -> float:
```

```
        # 自定义距离度量
        return float(np.linalg.norm(a - b))
```

```
    def cluster(self, Z: np.ndarray, **kwargs) -> tuple[np.ndarray, np.ndarray]:
```

```
        # 选择适合本领域的聚类方法
        from sklearn.cluster import KMeans
        n = kwargs.get("n_states", 10)
        km = KMeans(n_clusters=n, random_state=42)
        labels = km.fit_predict(Z)
```

```
    return labels, km.cluster_centers_
```

第 2 步：写 YAML（约 20-40 行）

```
# scx/domains/my_domain.yaml
domain: my_domain
description: " 我的领域数据价值评估"

encoder:
  class: scx.encoders.my_domain.MyDomainEncoder
  params:
    param1: 10
    param2: custom

experts:
  - name: expert_a
    type: model
    cost: 1.0
  - name: expert_b
    type: model
    cost: 5.0

state_config:
  n_states: 15
  cluster_method: kmeans

thresholds:
  error_high: 0.1
  noise_high: 0.4
```

```
    redundancy_high: 0.8
    expert_gap: 0.25

actions:
  - keep
  - compress
  - route
```

第 3 步：跑测试

测试脚本

```
from scx.core.domain_engine import SCXDomainEngine
from scx.domains.registry import DomainRegistry

# 1. 注册领域（自动或手动）
DomainRegistry.register("my_domain", "scx/domains/my_domain.yaml")

# 2. 创建引擎
engine = SCXDomainEngine("my_domain")

# 3. 加载数据
X, y = load_my_domain_data()

# 4. 运行 SCX pipeline
engine.fit(X, y)

# 5. 生成审计报告
report = engine.audit()
```

```
print(report.summary())

# 6. 查看结果
print(report.n_redundant, "redundant samples")
print(report.n_noisy, "noisy samples")
print(report.n_valuable, "valuable samples")
```

12.3 新增领域清单

#	任务	责任人	工作量估计
1	实现 SCXStateEncoder 接口	领域开发者	0.5-2 天
2	编写领域 YAML 配置	领域开发者	0.5 天
3	准备测试数据	领域开发者	0.5-2 天
4	运行测试	领域开发者	0.5 天
5	注册到 DomainRegistry	领域开发者	5 分钟

总计：约 2-5 人天（根据领域复杂度）

12.4 进阶：自定义 loss function

如果领域需要特殊损失函数（如力误差、物理约束违反），可以在 YAML 中指定：

```
# domains/custom_loss.yaml
loss_fn:
  module: my_module.losses
  function: my_custom_loss
```

```
params:
    alpha: 0.5
```

Adaapter 会自动加载并传入 ExpertReliability:

```
# core/adapter.py 内部
if "loss_fn" in domain_config:
    loss_cfg = domain_config["loss_fn"]
    module = importlib.import_module(loss_cfg["module"])
    loss_fn = getattr(module, loss_cfg["function"])
    if "params" in loss_cfg:
        loss_fn = partial(loss_fn, **loss_cfg["params"])
```

13. 数据流全景

用户输入

X (原始数据)
y (标签, 可选)
experts (专家列表, 可选)

SCXDomainEngine.fit(X, y)

1. Encoder.encode_batch(X) $\rightarrow$ Z (N, d)

2. `Encoder.cluster(Z) → labels, centroids`
3. `Build StateSpace.from(labels, centroids)`
→ `(s), D(s), C(s)`
4. `ExpertRegistry.predict_all(X) → Y_pred (M, N)`
5. `ExpertReliability.estimate(Y_pred, y, labels)`
→ `R_m(s), SCX_m(s), uncertainties`
6. `DataClassifier.classify_all(state_metrics)`
→ `Valuable / Redundant / Noisy / ExpertDependent`
7. `ActionPolicy.decide(classifications, V(s))`
→ `keep / compress / route / high-fidelity / ...`
8. `SCXAuditReport.generate(...)`
→ 结构化审计报告

SCXAuditReport

State Map: 每个状态的质心、半径、密度、样本数

Redundancy: 冗余状态列表、压缩比例建议

Noise Risk: 噪声状态列表、建议动作

High Value: 高价值状态列表、补数据/训练建议

Expert Reliability Matrix: (M, K) SCX 可靠性分数

Routing: 每个状态推荐的最佳专家
Recommendations: 下一轮数据/仿真/标注预算建议

14. 附录

A. 当前代码与新架构的映射

当前文件	新架构归属	迁移动作
core/framework.py	core/ + core/domain_engine.py	重构: SCXFramework 保持, 新增 SCXDomainEngine
core/config.py	core/config.py	扩展: 支持从 YAML 加载领域配置
core/online.py	core/online.py	不变, 也可作为插件
state/space.py	state/space.py	不变
expert/reliability.py	expert/reliability.py	不变
valuation/classifier.py	valuation/classifier.py	不变
action/policy.py	action/policy.py	不变
scx-health/src/scx_healthers/health.py	healthers/health.py	迁移并实现 SCXStateEncoder
experiments/cifar/utilencoders/vision.py	encoders/vision.py	CIFAR 部分迁移, 复 用统一接口
新增	encoders/base.py	新建
新增	encoders/mlip.py	新建
新增	encoders/trajectory.py	新建
新增	core/domain_engine.py	新建

当前文件	新架构归属	迁移动作
新增	core/adapter.py	新建
新增	core/audit.py	新建
新增	domains/registry.py	新建
新增	domains/*.yaml	新建（每个领域一个）

B. 完整用户入口示例

```
“python # ===== 方式 1：声明式（推荐）===== from
scx.core.domain_engine import SCXDomainEngine
```

一行加载领域配置

```
engine = SCXDomainEngine(“scx/domains/mlip.yaml”)
```

加载数据

```
X, y = load_dft_data() # 用户自己的数据加载
```

运行完整 pipeline

```
engine.fit(X, y)
```


获取结果

```
report = engine.audit() print(report)
```

压缩数据

```
X_sub, y_sub = engine.compress(X, y, ratio=0.5)
```

保存/加载

```
engine.save("outputs/my_project.scx") engine = SCXDomainEngine.load("outputs/my_project.scx")
```

===== 方式 2：编程式（灵活）

=====

```
from scx.encoders.mlip import ACEEncoder from scx.core.framework import  
SCXFramework from scx.core.config import SCXConfig
```

```
encoder = ACEEncoder(descriptor="soap", rcut=5.0) config = SCXCon-  
fig(n_states=20, n_experts=3) framework = SCXFramework(config=config)  
framework.fit(X, y, experts=expert_list, phi=encoder.encode) print(framework.summary())
```

===== 方式 3: 在线流式 =====

```
from scx.core.online import OnlineSCXFramework import numpy as np
```

初始质心

```
centroids = np.random.randn(10, 128) online = OnlineSCXFrame-
work(centroids, M=3)
```

```
for x, expert_id, loss in data_stream(): record = online.process_sample(x,
expert_id, loss) if record["classification"] == "noisy": print(f"Alert: noisy
sample at state {record['state']}")
```